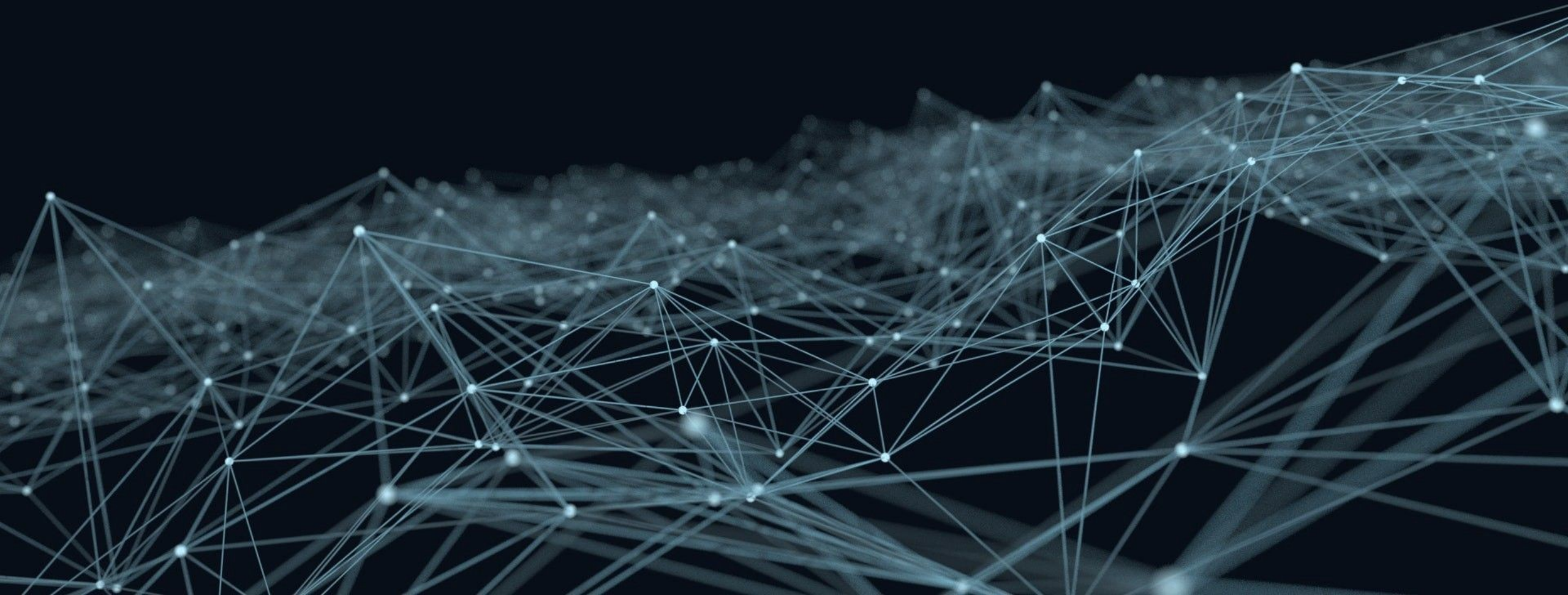
Java Full Stack Development

3. Secure Coding Practices



Input Validation

- Input validation
  - Validate at the boundary
    - The moment data enters your system
    - Controller, message handler, file reader
  - Validate by type, not by string content
    - Parse a date as LocalDate, not regex on a string
  - Prefer parsers over regex
    - Parsers are designed for the format
    - Regex catches some cases and misses others
  - Allowlist, not blocklist
    - Specify what's allowed, reject everything else
  - Centralize validation logic
    - One place to update when the rules change

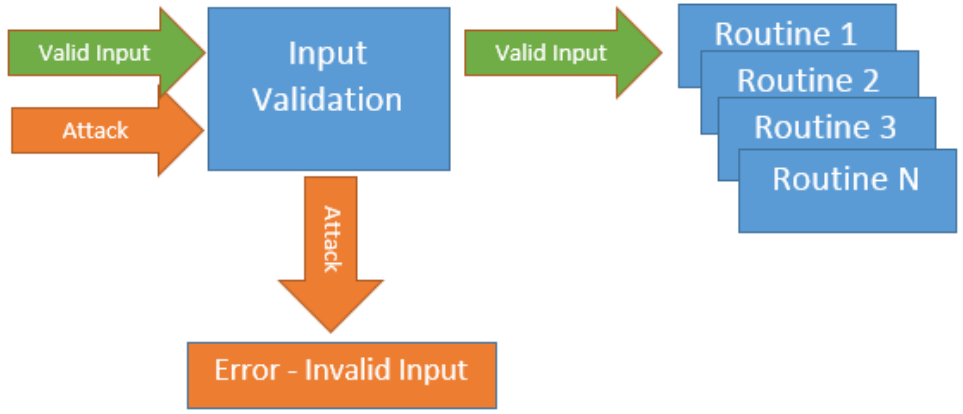


Image Credit: https://www.varutra.com/the-top-10-mistakes-to-avoid-when-reviewing-source-code/

Output Encoding

- Appropriate encoding at the output
  - Untrusted data is dangerous when interpreted by a downstream system
  - The right encoding depends on the destination context
    - HTML, JavaScript, URL, SQL, shell, log
  - Frameworks usually do this correctly by default
    - Manual configuration can break the defaults
    - Encode at the point of output, not at the point of input
    - HTML escape for HTML
    - Parameterize for SQL
    - URL-encode for URLs
    - JSON-encode for JSON

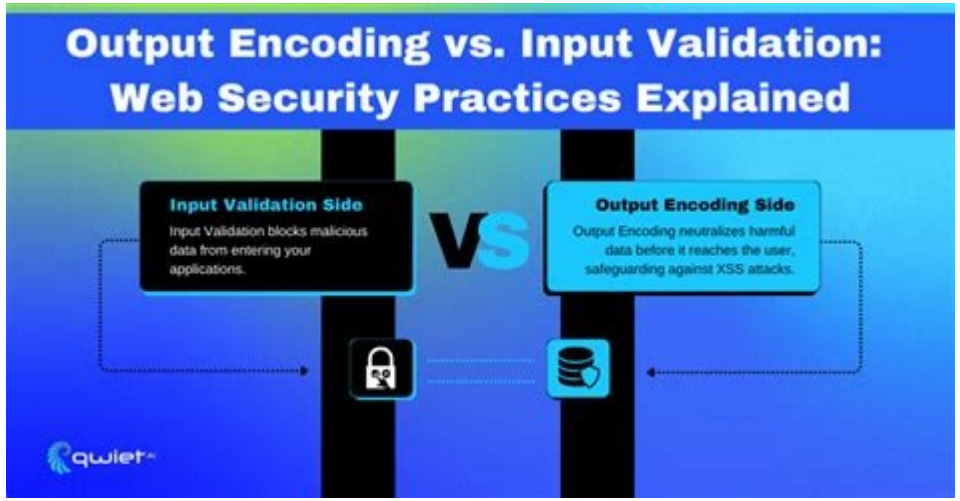


Image Credit: https://qwiet.ai/appsec-101-output-encoding/

Authentication vs. Authorization

- Not synonyms
- Authentication (AuthN)
  - "Who are you?"
  - Verifying identity
- Authorization (AuthZ)
  - "What are you allowed to do?"
  - Verifying permission

Image Credit: https://trevonix.com/blogs/authentication-vs-authorization/

Authentication vs. Authorization

- The flow we have seen
  - Authentication produces an identity
  - Authorization decides what that identity can do
  - A failed authentication is "401 Unauthorized"
    - Badly named because it means "unauthenticated"
  - A failed authorization is "403 Forbidden"
- Confusing them produces real security bugs

Image Credit: https://trevonix.com/blogs/authentication-vs-authorization/

Principle of Least Privilege

- Shows up everywhere
  - OAuth scopes
    - The OAuth client requests only the scopes it needs.
  - Database accounts
    - The application's database account has only the privileges its queries need
    - A read-only reporting app should have a SELECT-only account
    - Only the actual writers like controllers that insert/update data should use a higher-privilege account, and ideally a different one

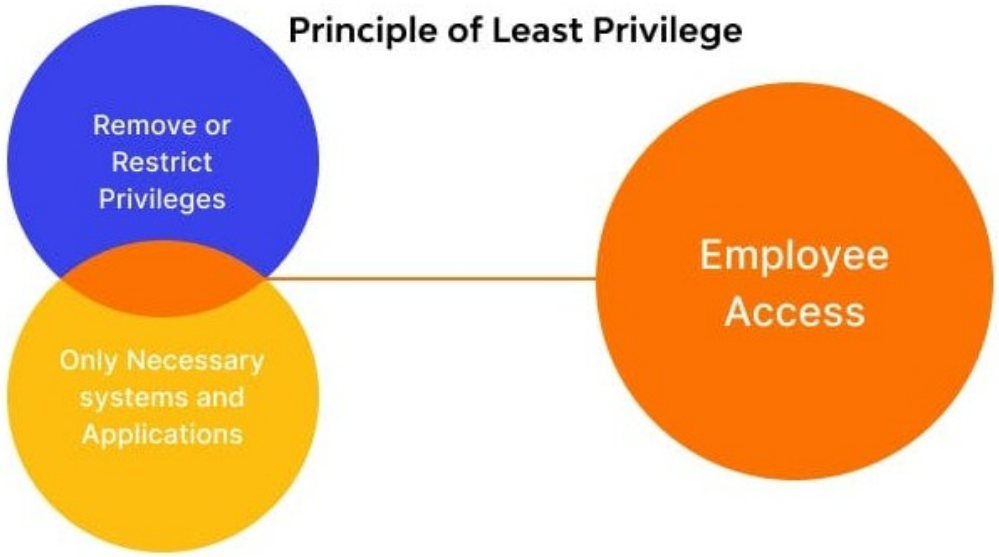


Image Credit: https://medium.com/@cloudmaterials01/principle-of-least-privilege-5d239a57a306

Principle of Least Privilege

- Shows up everywhere
  - Service accounts
    - When the BFF runs in production, it runs as a specific OS user
    - That user should not have admin privileges
    - It should not have write access to the application's installation directory
  - File system permissions
    - Files should be owned by users that need to read or write them, with permissions set accordingly
    - The configuration directory containing client secrets should be readable only by the service account

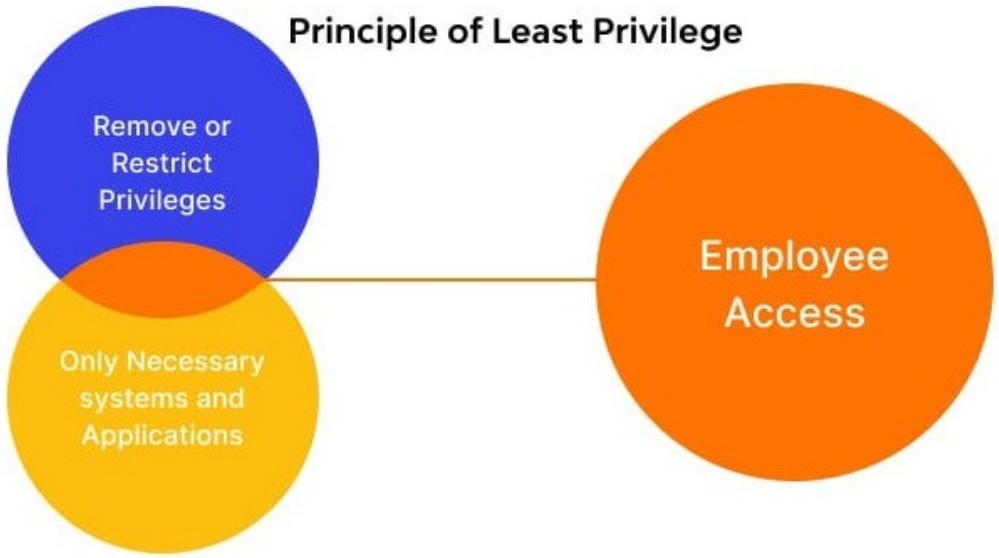


Image Credit: https://medium.com/@cloudmaterials01/principle-of-least-privilege-5d239a57a306

Principle of Least Privilege

- Shows up everywhere
  - Container capabilities
    - Docker containers can be configured with reduced Linux capabilities
    - A container running a Spring Boot app should not have ADMIN or other elevated capabilities unless required.
  - Network reachability
    - The BFF needs to reach the auth server and the resource server
    - It does NOT need to reach the entire internet
    - Network policies can restrict outbound traffic to only the required hosts

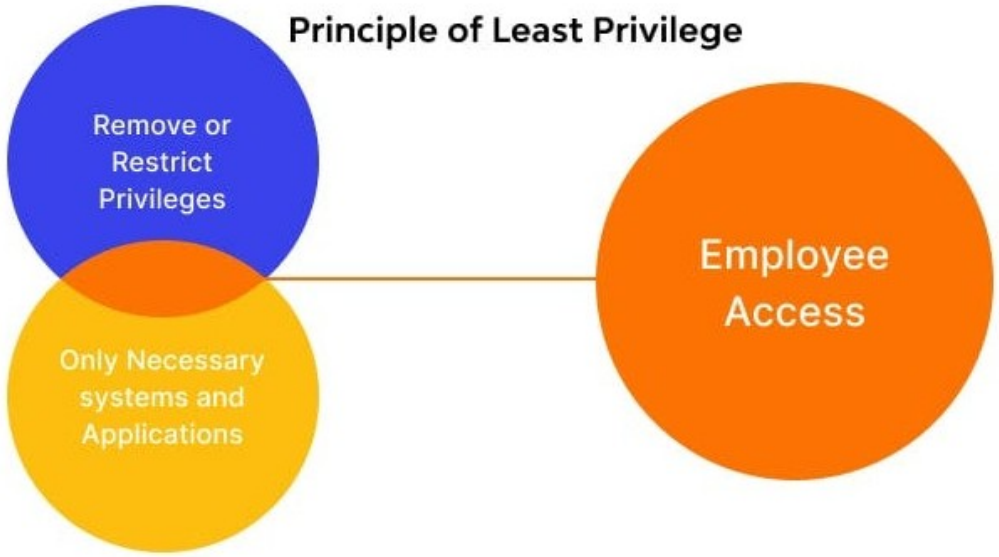


Image Credit: https://medium.com/@cloudmaterials01/principle-of-least-privilege-5d239a57a306

Defence in Depth

- Multiple risk controls
  - Never rely on a single security control
    - Each control reduces the probability of compromise; layered controls reduce it further
  - Examples
    - Authentication: OAuth + password complexity + (production) MFA
    - Network: HTTPS + firewall + WAF
    - Application: input validation + output encoding + parameterized queries
    - Detection: logging + monitoring + alerting
  - If one control fails, the others still hold

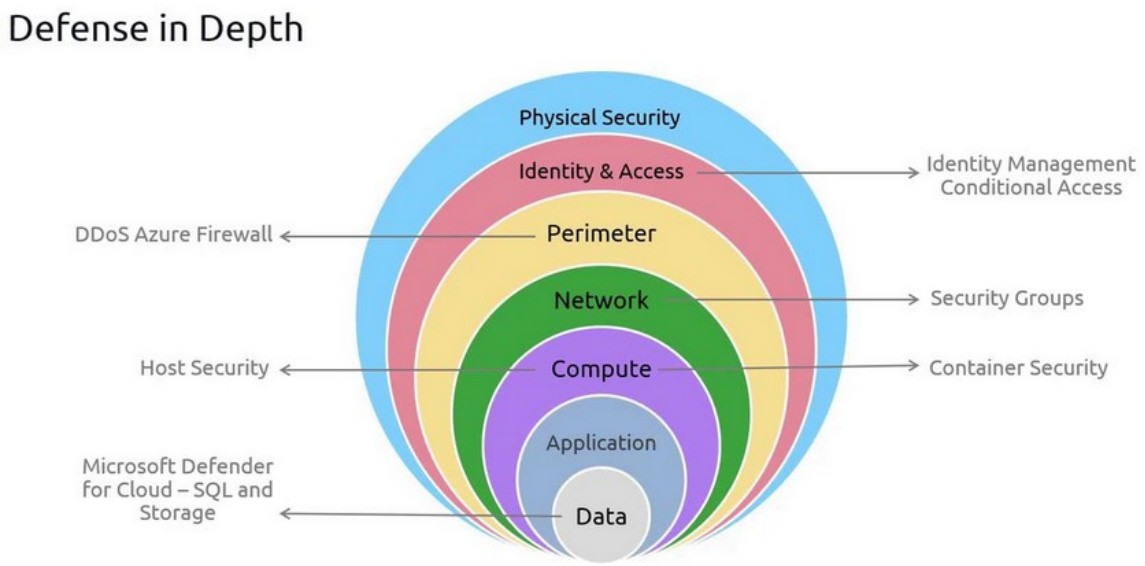


Image Credit: https://notes.kodekloud.com/docs/Microsoft-Azure-Security-Technologies-AZ-500/Container-Security/Introduction

Fail Securely

- When something goes wrong
  - Default to deny
  - The default state on error should be "deny," not "allow"
  - Common bug
    - Exception in authorization code is treated as "no rule matched"
    - So access granted
  - Correct pattern:
    - Exception in authorization
    - Log it and deny the request
  - Apply to
    - Error handlers, fallback logic, missing configuration, default value

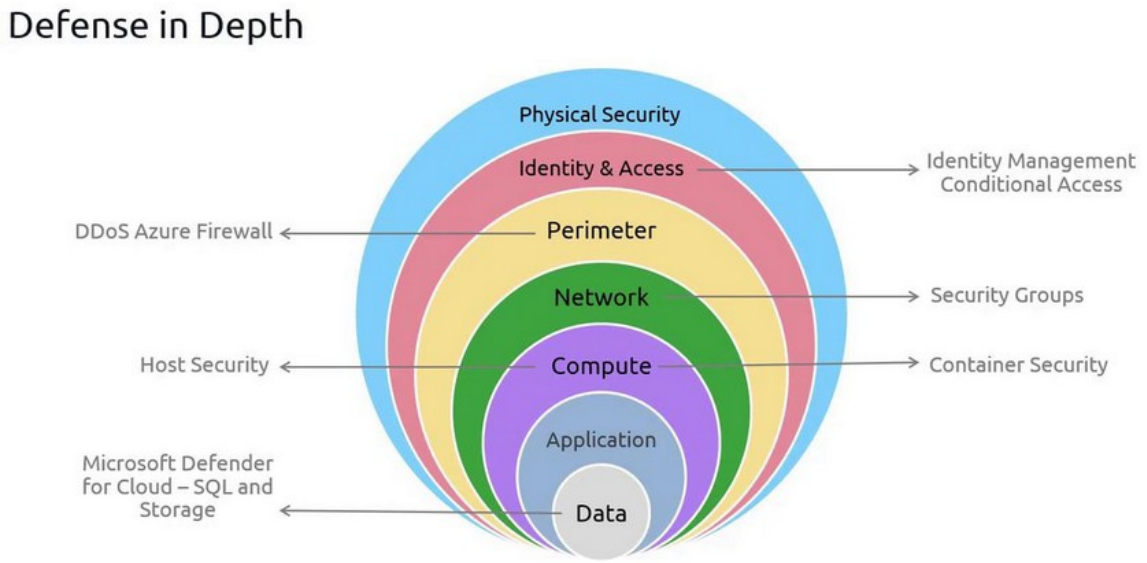


Image Credit: https://notes.kodekloud.com/docs/Microsoft-Azure-Security-Technologies-AZ-500/Container-Security/Introduction

OWASP Top Ten 2021

- Common security errors
  - A01: Broken Access Control
    - The most common category
    - Examples: a user can change a URL parameter to access another user's data
    - An admin endpoint isn't actually restricted to admins
    - The API trusts a "role" field sent by the client
    - SAST partial-detects; authorization bugs often require human review.

```
A01: Broken Access Control           <-- "users accessing things they shouldn't"
A02: Cryptographic Failures          <-- weak crypto, hardcoded keys, plaintext storage
A03: Injection                       <-- SQL, command, LDAP, XPath, etc.
A04: Insecure Design                 <-- design-level flaws, not just code bugs
A05: Security Misconfiguration       <-- defaults left on, debug enabled
A06: Vulnerable and Outdated Components <-- old libraries with known CVEs
A07: Identification/Auth Failures    <-- weak passwords, session fixation
A08: Software and Data Integrity Failures <-- unsigned updates, untrusted deserialization
A09: Security Logging/Monitoring Failures <-- can't detect attacks you don't log
A10: Server-Side Request Forgery     <-- code making requests to attacker-controlled URLs
```

OWASP Top Ten 2021

- Common security errors
  - A02: Cryptographic Failures
    - Using weak algorithms
    - Hhardcoded keys
    - Plaintext password storage
    - Plaintext sensitive data at rest
    - SAST detects most of these.
  - A03: Injection
    - SQL injection
    - Command injection
    - LDAP injection
    - XPath injection
    - SAST detects most of these.

```
A01: Broken Access Control           <-- "users accessing things they shouldn't"
A02: Cryptographic Failures          <-- weak crypto, hardcoded keys, plaintext storage
A03: Injection                       <-- SQL, command, LDAP, XPath, etc.
A04: Insecure Design                 <-- design-level flaws, not just code bugs
A05: Security Misconfiguration       <-- defaults left on, debug enabled
A06: Vulnerable and Outdated Components <-- old libraries with known CVEs
A07: Identification/Auth Failures    <-- weak passwords, session fixation
A08: Software and Data Integrity Failures <-- unsigned updates, untrusted deserialization
A09: Security Logging/Monitoring Failures <-- can't detect attacks you don't log
A10: Server-Side Request Forgery     <-- code making requests to attacker-controlled URLs
```

OWASP Top Ten 2021

- Common security errors
  - A04: Insecure Design
    - Architectural flaws that aren't fixable by code-level changes
    - A feature whose design is fundamentally insecure
    - Not SAST-detectable
    - Requires threat modeling and design review.
  - A05: Security Misconfiguration
    - Default credentials still active
    - Error messages revealing too much
    - Debug mode enabled
    - Security headers missing
    - Mix of SAST and DAST detection.

```
A01: Broken Access Control           <-- "users accessing things they shouldn't"
A02: Cryptographic Failures          <-- weak crypto, hardcoded keys, plaintext storage
A03: Injection                       <-- SQL, command, LDAP, XPath, etc.
A04: Insecure Design                 <-- design-level flaws, not just code bugs
A05: Security Misconfiguration       <-- defaults left on, debug enabled
A06: Vulnerable and Outdated Components <-- old libraries with known CVEs
A07: Identification/Auth Failures    <-- weak passwords, session fixation
A08: Software and Data Integrity Failures <-- unsigned updates, untrusted deserialization
A09: Security Logging/Monitoring Failures <-- can't detect attacks you don't log
A10: Server-Side Request Forgery     <-- code making requests to attacker-controlled URLs
```

OWASP Top Ten 2021

- Common security errors
  - A06: Vulnerable and Outdated Components
    - Using a library version with known CVEs. SCA
    - Dependency scanning tools catch this
  - A07: Identification and Authentication Failures
    - Weak password policies
    - Session fixation
    - Credential stuffing not protected
    - MFA not enforced
    - Mix of SAST and design review.

```
A01: Broken Access Control           <-- "users accessing things they shouldn't"
A02: Cryptographic Failures          <-- weak crypto, hardcoded keys, plaintext storage
A03: Injection                       <-- SQL, command, LDAP, XPath, etc.
A04: Insecure Design                 <-- design-level flaws, not just code bugs
A05: Security Misconfiguration       <-- defaults left on, debug enabled
A06: Vulnerable and Outdated Components <-- old libraries with known CVEs
A07: Identification/Auth Failures    <-- weak passwords, session fixation
A08: Software and Data Integrity Failures <-- unsigned updates, untrusted deserialization
A09: Security Logging/Monitoring Failures <-- can't detect attacks you don't log
A10: Server-Side Request Forgery     <-- code making requests to attacker-controlled URLs
```

OWASP Top Ten 2021

- Common security errors
  - A08: Software and Data Integrity Failures
    - Unsigned software updates
    - Deserializing untrusted data without integrity checks
    - Dependencies from untrusted sources
    - SAST detects unsafe deserialization
    - Supply chain integrity needs additional tooling.
  - A09: Security Logging and Monitoring Failures
    - Not logging security events
    - Not alerting on suspicious patterns
    - Hard to detect with tools
    - Requires audit and process review

```
A01: Broken Access Control           <-- "users accessing things they shouldn't"
A02: Cryptographic Failures          <-- weak crypto, hardcoded keys, plaintext storage
A03: Injection                       <-- SQL, command, LDAP, XPath, etc.
A04: Insecure Design                 <-- design-level flaws, not just code bugs
A05: Security Misconfiguration       <-- defaults left on, debug enabled
A06: Vulnerable and Outdated Components <-- old libraries with known CVEs
A07: Identification/Auth Failures    <-- weak passwords, session fixation
A08: Software and Data Integrity Failures <-- unsigned updates, untrusted deserialization
A09: Security Logging/Monitoring Failures <-- can't detect attacks you don't log
A10: Server-Side Request Forgery     <-- code making requests to attacker-controlled URLs
```

OWASP Top Ten 2021

- Common security errors
  - A10: Server-Side Request Forgery (SSRF)
    - Application making HTTP requests to URLs controlled by attackers
    - Allowing access to internal resources
    - SAST partial-detects.

```
A01: Broken Access Control           <-- "users accessing things they shouldn't"
A02: Cryptographic Failures          <-- weak crypto, hardcoded keys, plaintext storage
A03: Injection                       <-- SQL, command, LDAP, XPath, etc.
A04: Insecure Design                 <-- design-level flaws, not just code bugs
A05: Security Misconfiguration       <-- defaults left on, debug enabled
A06: Vulnerable and Outdated Components <-- old libraries with known CVEs
A07: Identification/Auth Failures    <-- weak passwords, session fixation
A08: Software and Data Integrity Failures <-- unsigned updates, untrusted deserialization
A09: Security Logging/Monitoring Failures <-- can't detect attacks you don't log
A10: Server-Side Request Forgery     <-- code making requests to attacker-controlled URLs
```

Secure Logging

- Logs: common source of breaches
  - Log security-relevant events
    - Logins, logouts, authorization failures, configuration changes, admin actions
  - Never log secrets
    - Passwords, tokens, full credit card numbers, social security numbers, API keys
  - Beware of log injection
    - Untrusted data in log messages can corrupt log files or trick log analysis tools
  - Use structured logging
    - JSON or key-value pairs, not free-form text
  - Logs are sensitive
    - Restrict who can read them, retain them per policy, encrypt at rest

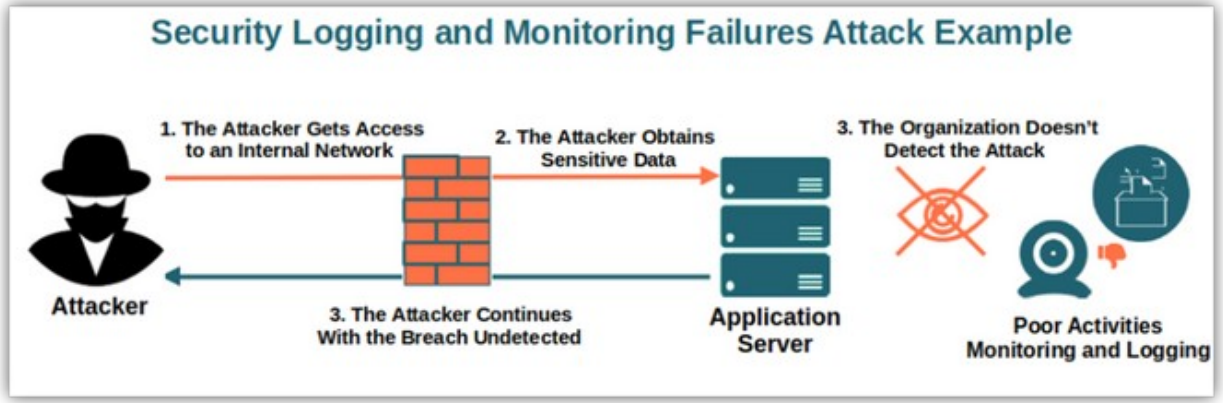


Image Credit: https://www.jit.io/resources/security-standards/the-in-depth-guide-to-owasps-top-10-vulnerabilities

Secure Logging

- Log files are routinely
  - Stored on disk for weeks or months
  - Replicated to centralized logging systems
  - Indexed and searchable
  - Available to operations staff, support staff, and sometimes developers
  - Backed up to long-term storage
  - Sometimes shared with third-party log analysis vendors

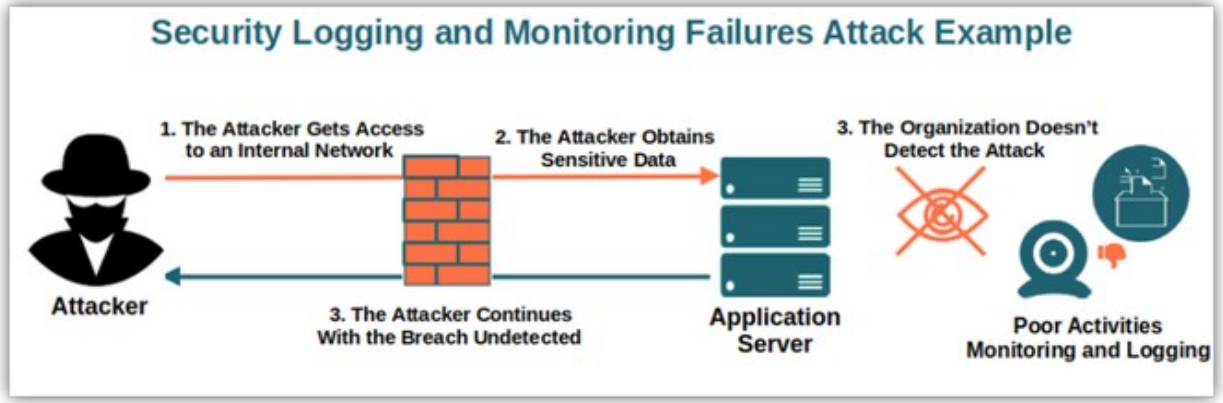


Image Credit: https://www.jit.io/resources/security-standards/the-in-depth-guide-to-owasps-top-10-vulnerabilities

Secure Logging

- What to log
  - Authentication events
    - Login success, login failure, logout
  - Authorization decisions
    - Especially failures
  - Configuration changes
  - Administrative actions
  - Errors and exceptions
  - Performance metrics for security-relevant operations

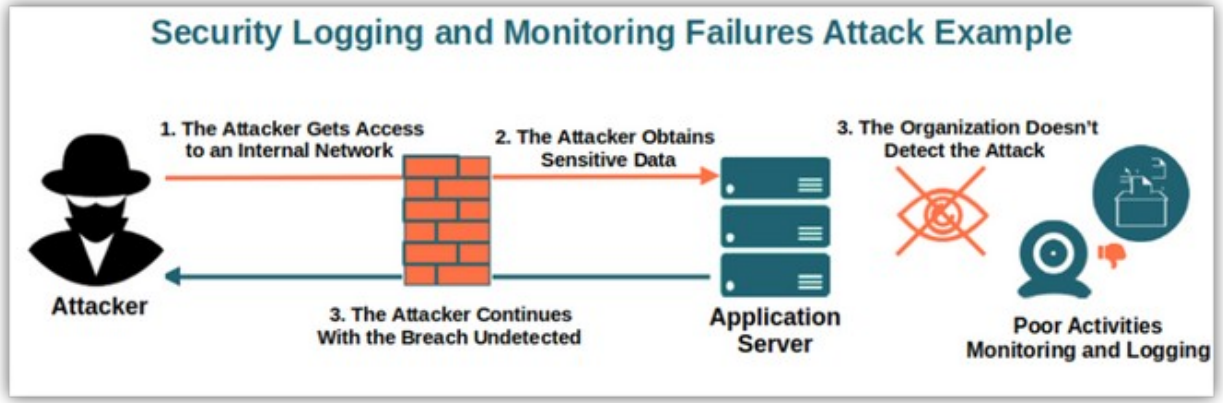


Image Credit: https://www.jit.io/resources/security-standards/the-in-depth-guide-to-owasps-top-10-vulnerabilities

Secure Logging

- What never to log
  - Passwords, in any form
    - Plaintext, hashed, encrypted
  - OAuth access tokens, refresh tokens, ID tokens
  - Session cookies or session IDs
  - Credit card numbers
  - Social Security numbers, government IDs
  - Private encryption keys
  - Personal health information
    - HIPAA scope
  - Anything else covered by data protection regulations

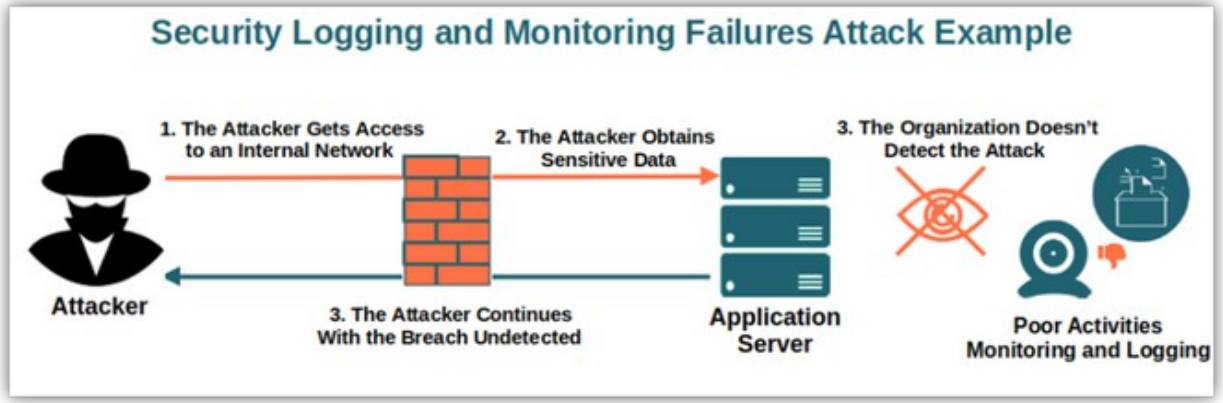


Image Credit: https://www.jit.io/resources/security-standards/the-in-depth-guide-to-owasps-top-10-vulnerabilities

Dependency Management

- A typical Spring Boot application has
  - 100+ direct dependencies and 1000+ transitive ones
  - Each dependency is a potential vulnerability vector
  - Keep dependencies current
    - Old versions accumulate known vulnerabilities
  - Use SCA tools
    - Automated detection of vulnerable dependencies
  - Understand transitive dependencies
    - You don't directly use most of them, but they run in your process
  - Pin versions
    - Know exactly what you're shipping
    - Reproducible builds

Image Credit: https://betanews.com/article/amusing-bsods-from-around-the-world-following-crowdstrikes-disastrous-windows-outage/

Shift Left

- Shift-left
  - Find security issues earlier in the SDLC
    - Where they're cheaper to fix
  - IDE plugins flag issues as you type
  - SAST in CI catches what you missed
  - Pre-deploy DAST catches runtime issues

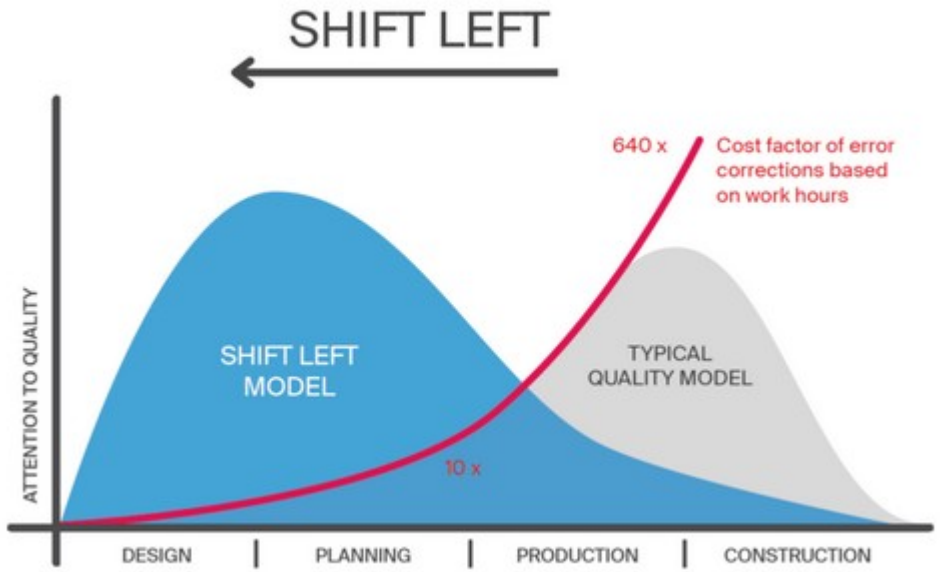


Image Credit: https://storage.googleapis.com/deymjzwhtkbyye/shift-left-process.html

Questions?